

The background of the slide features a light blue and white DNA double helix structure, with the helix appearing to spiral around the central text. The helix is composed of two intertwined strands connected by horizontal rungs, creating a sense of depth and movement.

MP DNA Forensics

Motivation

- DNA analysis is important to a variety of applications, such as criminal investigations, biometric verification, and human identification.
- Determining who a specific DNA strand belongs to is tedious, so regions called Short Tandem Repeats (STRs) are often used to match a genome to a person.
- In this MP you will: divide and conquer with functions, parse strings, practice input file streams, and execute command line arguments.



Background

- There are **four nitrogenous bases** in DNA: Adenine (A), Thymine (T), Guanine (G), and Cytosine (C).
 - We will represent them in our code as the characters **'A' 'T' 'G' 'C'**.
- **Short Tandem Repeats (STRs)** are short DNA sequences that repeat consecutively.
 - We will match a person to DNA by their counts of specific STRs.
 - We only care about the **longest consecutive sequence** when matching.

ATTAATTAATTAATTAATGAATGAATGTATTAATTAATTAATCTA

Counts for the above strand: ATTA → 4 GAAT → 2

Simple Example

Person 1: ACG→3, GTTA→4

Person 2: ACG→1, GTTA→2

Person 3: ACG→5, GTTA→2



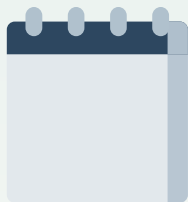
GCTACGTAGTTAGTTATGAACTCA

This DNA matches with Person 2

Outlined Process



Command line
arguments



CSV file



ProfileDNA

...

Your program



Was there a
match?

Inputs

- **Command Line Arguments**

- your program will be run with arguments that specify the DNA Database File and the DNA strand to be analyzed.

e.g. `./bin/exec data.csv ATTAATTAATTAATTAAATGAATGA`

- **DNA Database File**

- CSV file
- Contains each person and their STR counts
- Assume there is at least one STR and one person

e.g.

```
Names,AGATG,AATG,TAT
Casey,5,2,8
Amani,3,7,4
Blair,6,1,5
```

DNA Database File

```
Nombres,GTCA,ATG,TTCA,GGTC  
Bianca,3,5,7,1  
Justin,6,3,1,2  
Reyna,7,3,2,4  
Mary,6,1,2,9  
Ricardo,2,3,5,2
```

- There is at least one STR and person in the database.
- The first row serves as the column names of the database.
- The **first column** will be names, though it can be called anything.
 - (i.e. Names, Nombres, etc).
- You will need to use a `std::ifstream` to read in the file's contents.

ProfileDNA

- Main function to implement (in `functions.cc`)
- Inputs (`std::string`): 1) `dna_database` 2) `dna_sequence`
 - You will call `ProfileDNA` from the `main` function using the command line args.
- Returns the person's name as a string who matches `dna_sequence`.
 - May also return "No match" (case sensitive).

There is no match if:

- None of the people have STR counts that match the DNA strand.
- More than one person matches the DNA strand.

More Examples

Person 1: ATTA → 1, GC → 2, TAT → 2

Person 2: ATTA → 1, GC → 2, TAT → 2



ATTATTATAACCCTGCGCGGATCC

ATTATTATAACCCTGCGCGGATCC

ATTATTATAACCCTGCGCGGATCC



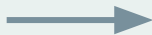
*Same strand, shown
3 times*

Return “No match” because this DNA strand matches
Person 1 AND Person 2.

More Examples



GCTAGAGAAGAAGAACTCA



Correct parsing for the STR “AGA”:

GCTAGAGAAGAAGAACTCA



GCTAGAGAAGAAGAACTCA



You **cannot reuse characters** from an earlier match to create another match for the same STR (cannot overlap characters).

GCTAGAGAAGAAGAACTCA

... scan the strand from left to right.

Helpful Tips

- Start early & test your code locally (make use of the [debugger!](#))
- Use the provided function `utilities::GetSubstrs` to help parse the CSV file.
- Make sure to handle any edge cases (like no matches, STR overlap, etc).
- Break up your logic into helper functions. Each function should perform one main task.
- You may only **#include** the following headers:

```
<cassert> <fstream> <iostream> <map> <stdexcept> <string> <vector> "functions.hpp" "utilities.hpp"
```

Important: Use LF (Linux) Line Endings

To ensure your code works correctly with our tests, your test files **MUST** use **LF** line endings, not Windows' **CRLF**.

To confirm that your test files are using **LF** endings, in Visual Studio Code, check the bottom-right corner of the window. You should see either **LF** or **CRLF**.

If it says **CRLF**: Click it and select **LF** to change the encoding.

If it says **LF**: Your file is correctly configured.

Please make this change for all your input files.

Any Questions?

Good luck!